Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

Hands-On #7

Socket Multicast UDP

Last Update: March 26, 2026

Scenario

Hai già scritto client e server TCP nei laboratori precedenti. Quella architettura funziona bene per la comunicazione uno-a-uno, ma ha un limite strutturale: per far comunicare N peer tra loro senza un server centrale servono $N \times (N-1)/2$ connessioni TCP simultanee, una per ogni coppia. Con 10 peer sono 45 connessioni; con 100 peer sono 4950.

L'alternativa con server centrale riduce le connessioni a N, ma introduce un collo di bottiglia: il server deve ricevere ogni messaggio e rispedirlo a tutti gli altri $N-1$ peer. All'aumentare dei partecipanti il server diventa il punto più lento del sistema.

Il **multicast** risolve entrambi i problemi: il mittente fa una sola `sendto()` e la rete si occupa di consegnare il pacchetto a tutti gli host che si sono iscritti al gruppo, senza server centrale e senza moltiplicare il lavoro del mittente.

In questo laboratorio partirai da un sender e un receiver già funzionanti, esplorerai il comportamento del multicast attraverso esperimenti diretti, e arriverai a costruire una chat di gruppo introducendo un problema alla volta.

Concetti chiave

Il gruppo multicast

Un gruppo multicast è identificato da un indirizzo IP nel blocco **224.0.0.0/4**. In laboratorio usiamo sempre il range **239.0.0.0/8** (*administratively scoped*): per convenzione i pacchetti con questi indirizzi non vengono instradati fuori dalla rete locale, analogamente al ruolo delle subnet private 192.168.x.x per l'unicast.

Un host si iscrive a un gruppo con `setsockopt(IP_ADD_MEMBERSHIP)`. Da quel momento il kernel consegna al socket tutti i datagrammi UDP destinati a quell'indirizzo di gruppo su quella porta. Chi invia non deve iscriversi: basta fare `sendto()` verso l'indirizzo del gruppo.

Il loopback multicast

Per default, quando un processo invia un pacchetto multicast, il kernel ne consegna una copia anche ai socket locali iscritti allo stesso gruppo — senza che il pacchetto esca sulla rete fisica. Questo comportamento si chiama *multicast loopback* ed è controllato dall'opzione `IP_MULTICAST_LOOP` (abilitata per default). È molto comodo per i test su una sola macchina.

Il problema del blocco e select()

`recvfrom()` è una chiamata *bloccante*: se non arriva nessun datagramma, il processo si ferma lì e non fa nient'altro. Lo stesso vale per `fgets()` su `stdin`: se l'utente non scrive nulla, il processo si ferma ad aspettare l'input.

Mettere le due chiamate in sequenza non funziona: mentre aspetti sulla rete non puoi leggere da tastiera, e viceversa.

La soluzione è `select()`, che permette di aspettare su più file descriptor contemporaneamente:

```
int select(int nfds,
           fd_set *readfds,    /* fd da monitorare in lettura */
           fd_set *writefds,   /* fd da monitorare in scrittura (
               NULL) */
           fd_set *exceptfds, /* eccezioni (NULL) */
           struct timeval *timeout); /* NULL = aspetta all'infinito
               */
```

`select()` blocca finché *almeno uno* dei file descriptor in `readfds` diventa pronto per la lettura, poi ritorna. A quel punto controlli quale fd si è svegliato e lo gestisci.

[!] Attenzione

`select()` **modifica** `readfds` in-place: al ritorno, la `fd_set` contiene solo i fd effettivamente pronti, non tutti quelli che avevi impostato. Devi quindi ripetere `FD_ZERO` e `FD_SET` all'inizio di **ogni** iterazione del loop.

Le macro per gestire una `fd_set`:

```
fd_set readfds;
FD_ZERO(&readfds);              /* azzera tutti i bit */
FD_SET(sockfd, &readfds);       /* aggiungi sockfd al set */
FD_SET(STDIN_FILENO, &readfds); /* aggiungi stdin al set */

/* nfds = valore massimo dei fd nel set + 1 */
select(sockfd + 1, &readfds, NULL, NULL, NULL);

if (FD_ISSET(sockfd, &readfds))      { /* sockfd e' pronto */ }
if (FD_ISSET(STDIN_FILENO, &readfds)){ /* stdin e' pronto */ }
```

Codice di partenza

I due programmi seguenti sono completi e funzionanti. Leggili con attenzione, poi compilali ed eseguili prima di procedere con gli esercizi.

Listing 1: sender.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MCAST_GROUP "239.0.0.1"
#define MCAST_PORT   5000

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <messaggio>\n", argv[0]);
        return 1;
    }

    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) { perror("socket"); return 1; }
```

```c

    /* TTL = 1: il pacchetto non esce dalla LAN */
    int ttl = 1;
    setsockopt(sockfd, IPPROTO_IP, IP_MULTICAST_TTL,
               &ttl, sizeof(ttl));

    struct sockaddr_in dest;
    memset(&dest, 0, sizeof(dest));
    dest.sin_family = AF_INET;
    dest.sin_port   = htons(MCAST_PORT);
    inet_pton(AF_INET, MCAST_GROUP, &dest.sin_addr);

    const char *msg = argv[1];
    ssize_t sent = sendto(sockfd, msg, strlen(msg), 0,
                          (struct sockaddr *)&dest, sizeof(dest));
    if (sent < 0) { perror("sendto"); return 1; }

    printf("Inviato (%zd byte): \"%s\"\n", sent, msg);
    close(sockfd);
    return 0;
}
```

Listing 2: receiver.c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MCAST_GROUP "239.0.0.1"
#define MCAST_PORT   5000
#define BUFLEN       1024

int main(void) {
    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) { perror("socket"); return 1; }

    /* SO_REUSEADDR: permette a piu' processi di fare bind
       sulla stessa porta nella stessa macchina */
    int yes = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR,
               &yes, sizeof(yes));

    /* bind sull'indirizzo del gruppo: il kernel filtra qui
       i datagrammi diretti ad altri gruppi sulla stessa porta */
    struct sockaddr_in local;
    memset(&local, 0, sizeof(local));
    local.sin_family = AF_INET;
    local.sin_port   = htons(MCAST_PORT);
    inet_pton(AF_INET, MCAST_GROUP, &local.sin_addr);

    if (bind(sockfd, (struct sockaddr *)&local,
             sizeof(local)) < 0) {
        perror("bind"); return 1;
    }

    /* Iscrizione al gruppo: il kernel manda un IGMP Join */
```

```c
    struct ip_mreq mreq;
    inet_pton(AF_INET, MCAST_GROUP, &mreq.imr_multiaddr);
    mreq.imr_interface.s_addr = htonl(INADDR_ANY);
    setsockopt(sockfd, IPPROTO_IP, IP_ADD_MEMBERSHIP,
               &mreq, sizeof(mreq));

    printf("In ascolto su %s porta %d...\n",
           MCAST_GROUP, MCAST_PORT);

    char buf[BUFLEN];
    struct sockaddr_in sender;
    socklen_t senderlen = sizeof(sender);

    while (1) {
        ssize_t n = recvfrom(sockfd, buf, sizeof(buf) - 1, 0,
                             (struct sockaddr *)&sender,
                             &senderlen);
        if (n < 0) { perror("recvfrom"); break; }
        buf[n] = '\0';

        /* inet_ntop e' thread-safe; inet_ntoa non lo e' */
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &sender.sin_addr,
                  ip, sizeof(ip));
        printf("[%s:%d] %s\n", ip, ntohs(sender.sin_port), buf);
    }

    setsockopt(sockfd, IPPROTO_IP, IP_DROP_MEMBERSHIP,
               &mreq, sizeof(mreq));
    close(sockfd);
    return 0;
}
```

▷ Come compilare ed eseguire

```
gcc sender.c   -o sender   -Wall -Wextra
gcc receiver.c -o receiver -Wall -Wextra

# Terminale 1:
./receiver

# Terminale 2:
./sender "ciao a tutti"
```

Su una sola macchina funziona grazie al loopback multicast del kernel. Per testare su due macchine nella stessa LAN non serve nessuna modifica.

Esercizi

Esercizio 1 — Osserva il comportamento base

1. Compila ed esegui sender e receiver. Apri **due terminali** con `./receiver` in esecuzione simultaneamente, poi invia un messaggio con `./sender`. Quanti receiver lo ricevono? Perché questo non sarebbe possibile con TCP unicast?
2. Avvia `sudo tcpdump -i any udp port 5000 -e` in un terminale separato, poi invia

un messaggio. Osserva il campo MAC di destinazione nel frame Ethernet catturato. Confrontalo con la formula `01:00:5e:XX:XX:XX` dove XX:XX:XX derivano dagli ultimi 23 bit dell'IP del gruppo. Il MAC che vedi corrisponde?

3. Modifica `MCAST_GROUP` in `sender.c` e `receiver.c` sostituendo `239.0.0.1` con `239.1.2.3`, ricompila ed esegui. Cattura di nuovo con `tcpdump`: qual è il nuovo MAC di destinazione?

▷ Suggerimento

Il MAC multicast si calcola così: prendi gli ultimi 23 bit dell'indirizzo IP, mettili nei 23 bit bassi di `01:00:5e:00:00:00`. Per `239.1.2.3`: i tre ottetti bassi sono `01`, `02`, `03` → MAC `01:00:5e:01:02:03`.

Esercizio 2 — TTL e loopback

1. Nel `sender.c` cambia `ttl = 1` in `ttl = 0` e ricompila. Cosa succede quando esegui `./sender "test"`? Perché?
2. Ripristina `ttl = 1`. Ora aggiungi questa riga nel `sender.c` subito dopo quella del TTL:

```
int loop = 0;
setsockopt(sockfd, IPPROTO_IP, IP_MULTICAST_LOOP,
           &loop, sizeof(loop));
```

   Ricompila ed esegui sender e receiver sulla **stessa macchina**. Il receiver riceve ancora il messaggio? Prova ad avviare il receiver su una macchina diversa nella stessa LAN: cambia qualcosa?
3. Spiega con parole tue la differenza tra `IP_MULTICAST_TTL` e `IP_MULTICAST_LOOP`: cosa controlla ciascuna opzione e a quale livello agisce?

[!] Attenzione

`IP_MULTICAST_LOOP` controlla se il kernel riconsegna il pacchetto ai socket locali **prima** che esca sulla rete. `IP_MULTICAST_TTL` controlla quanti router il pacchetto può attraversare **sulla rete**. Sono due meccanismi completamente indipendenti.

Esercizio 3 — Costruisci una chat multicast

Fino ad ora sender e receiver sono due programmi separati. L'obiettivo di questo esercizio è costruire un unico programma `chat.c` in cui ogni peer può sia inviare che ricevere messaggi dal gruppo. Segui i passi nell'ordine indicato.

Passo A — Un solo socket per inviare e ricevere

Prima osservazione: non servono due socket distinti. Un socket UDP su cui hai fatto `bind()` e `IP_ADD_MEMBERSHIP` può sia ricevere con `recvfrom()` che inviare con `sendto()` verso l'indirizzo del gruppo. Il socket è simmetrico.
Crea `chat.c` copiando la struttura di `receiver.c` (bind + IP_ADD_MEMBERSHIP) e aggiungi in fondo al loop, *prima* di `recvfrom()`, una `sendto()` di prova verso il gruppo. Verifica che il messaggio inviato appaia anche in ricezione (grazie al loopback).

Passo B — Il problema del blocco

Ora prova a mettere le due operazioni in sequenza nel loop:

```
while (1) {
```

```
    /* leggi una riga da tastiera... */
    fgets(buf, sizeof(buf), stdin);
    sendto(sockfd, buf, strlen(buf), 0,
           (struct sockaddr *)&dest, sizeof(dest));

    /* ...poi aspetta un messaggio dalla rete */
    recvfrom(sockfd, buf, sizeof(buf)-1, 0, ...);
    printf("Ricevuto: %s\n", buf);
}
```

Esegui il programma su due terminali e osserva il problema: mentre aspetti che l'utente digiti qualcosa, il programma non può ricevere messaggi dalla rete e viceversa. Questo è il blocco su I/O sequenziale.

Passo C — select() come soluzione

Sostituisci il loop con uno basato su `select()`. L'idea è: invece di chiamare direttamente `fgets()` o `recvfrom()`, chiedi prima al kernel quale dei due file descriptor è pronto, e poi chiama solo quello:

```
while (1) {
    /* FD_ZERO e FD_SET vanno ripetuti ad ogni iterazione:
       select() modifica readfds in-place, azzerando i bit
       dei fd che non si sono svegliati.                    */
    fd_set readfds;
    FD_ZERO(&readfds);
    FD_SET(STDIN_FILENO, &readfds); /* tastiera */
    FD_SET(sockfd, &readfds);       /* rete */

    /* blocca finche' almeno uno dei due e' pronto */
    select(sockfd + 1, &readfds, NULL, NULL, NULL);

    if (FD_ISSET(STDIN_FILENO, &readfds)) {
        /* l'utente ha premuto Invio: leggi e invia */
        fgets(buf, sizeof(buf), stdin);
        /* rimuovi il newline finale */
        buf[strcspn(buf, "\n")] = '\0';
        if (strlen(buf) > 0)
            sendto(sockfd, buf, strlen(buf), 0,
                   (struct sockaddr *)&dest, sizeof(dest));
    }

    if (FD_ISSET(sockfd, &readfds)) {
        /* e' arrivato un datagramma: ricevi e stampa */
        ssize_t n = recvfrom(sockfd, buf, sizeof(buf)-1, 0,
                             (struct sockaddr *)&sender,
                             &senderlen);
        buf[n] = '\0';
        inet_ntop(AF_INET, &sender.sin_addr,
                  ip, sizeof(ip));
        printf("[%s] %s\n", ip, buf);
    }
}
```

Implementa questo loop in `chat.c` e verifica che ricevere e inviare funzionino contemporaneamente senza bloccarsi.

Passo D — Il proprio messaggio torna indietro

Eseguendo la chat ti accorgerai che ogni messaggio inviato appare anche come messaggio

ricevuto — perché `IP_MULTICAST_LOOP` è abilitato per default. Risolvi il problema in uno dei due modi seguenti:

a) Aggiungi il nome utente (passato come `argv[1]`) al payload nel formato `[nome] testo`. All'arrivo, controlla se il nome nel messaggio ricevuto corrisponde al proprio: in caso affirmativo non stamparlo.

b) In alternativa, disabilita il loopback con `IP_MULTICAST_LOOP = 0` e osserva cosa cambia. Qual è la differenza tra le due soluzioni?

▷ Struttura finale di chat.c

1. `socket()` + `SO_REUSEADDR`
2. `bind()` sull'indirizzo del gruppo
3. `IP_ADD_MEMBERSHIP`
4. `IP_MULTICAST_TTL = 1`
5. Loop con `select()` su `STDIN_FILENO` e `sockfd`
6. Su `SIGINT`: `IP_DROP_MEMBERSHIP` e `close()`

[!] Uscita pulita

Installa un gestore per `SIGINT` con `signal(SIGINT, handler)` in modo da chiamare `IP_DROP_MEMBERSHIP` prima di chiudere. Senza di esso il kernel manda comunque un IGMP Leave alla chiusura del socket, ma farlo esplicitamente è buona norma e ti permette di aggiungere altri cleanup in futuro.